Verification Course Exercises

SyoSil Aps ©

2026

Contents

Table of Contents **2**

1 Introduction **3**

2 Design of the Saturation Filter **3**
2.1 Implementation 3

3 Simple SyoSil Data Transfer Protocol **5**

4 *sSDT* Universal Verification Component **6**

A Appendix **7**

1 Introduction

This document contains information on the SAT Filter, relating topics and general exercise information. It will be updated throughout the course with new information, whenever relevant.

2 Design of the Saturation Filter

A *Saturation Filter* is a device that saturates the input when this exceeds certain limits. The design provided implements this device and will be used as the DUT (Device Under Test) for the following exercises. The device saturates the input data when this exceeds the threshold value defined.

2.1 Implementation

The Figure 2.1 shows the high-level diagram of the *Saturation Filter*, showing the input and output ports of the design.

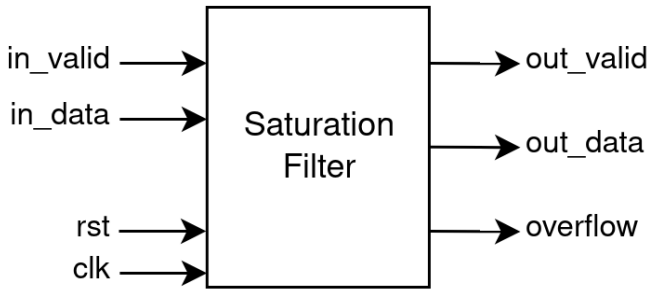


Fig. 2.1: *Saturation Filter* module.

The *Saturation Filter* I/O ports are the following:

- `clk`, the clock signal;
- `rst`, the reset signal;
- `in_valid`, the input valid signal;
- `in_data`, the input data sample;
- `out_valid`, the output valid signal;
- `out_data`, the output data sample;
- `overflow`, the output overflow signal.

The *Saturation Filter* module contains the following parameters. They are set to a default value and should be kept as such.

- `DATA_W`, sets the data width of the data signal; The default value is 8.
- `THRESHOLD`, the value over which to saturate the data. The default value is 64.

The device uses the SyoSil Data Transfer Protocol (*sSDT*) Section 3, and was developed respecting the following requirements:

- **RS/01**: It shall be possible to reset the state of the saturation filter by toggling the `rst` input signal.
- **RS/02**: All signals shall react to the rising edge of the input `clk` signal.
- **RS/03**: The saturation functionality of the design shall be evaluated with the `THRESHOLD` parameter value. If the data is below the limits it is propagated to the output on the next rising edge of the `clk` signal. Otherwise, the data is `saturated` and the `out_data` signal will be the `THRESHOLD` value instead.
- **RS/04**: The `in_valid` signal is always propagated to the output on the next rising edge of the `clk`.
- **RS/05**: The *Saturation Filter* shall be compliant with the *sSDT* (simple SyoSil Data Transfer) protocol described in the following section.

3 Simple SyoSil Data Transfer Protocol

The simple SyoSil Data Transfer (*sSDT*) protocol is a simple synchronous data transfer protocol. The protocol has two signals, data and valid, and has two variants, the "producer" and the "consumer", which are modifying the direction of the signal as showed in the Table 3.1.

Signal Name	Producer direction	Consumer direction	Comment
valid	output	input	when asserted, data is valid
data	output	input	the data of the protocol

Table 3.1: *sSDT* protocol signals.

The protocol operation must follow these protocol requirements (PR):

1. **Reset requirement (PR1)**: when `rst` is high, `valid` can not be high
2. **Data validity requirement (PR2)**: when `vaild` is high, `data` must have a valid value
3. **Data invalidity requirement (PR3)**: when `valid` is low, `data` must be `0`

A timing diagram outlining the behavior of the protocol is shown in Figure 3.1. All requirements should be valid when checked synchronously to the clk.

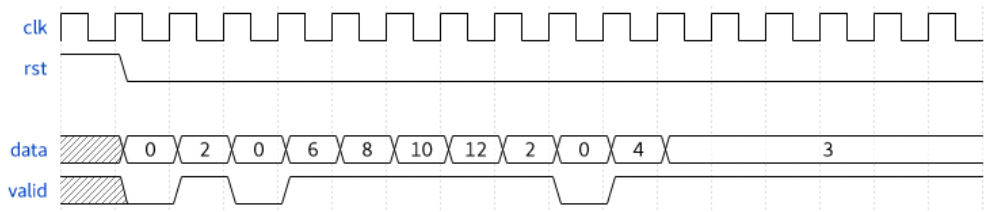


Fig. 3.1: *sSDT* protocol waveform.

4 *sSDT* Universal Verification Component

The *sSDT* Universal Verification Component (*UVC*) is implemented with the intention of generating traffic compliant with the protocol requirements. By developing the *sSDT UVC* it is ensured the reuse of the same component in multiple testbenches, without having to re-implement the code from the beginning. The Figure 4.1 shows the high-level diagram of the *sSDT UVC*.

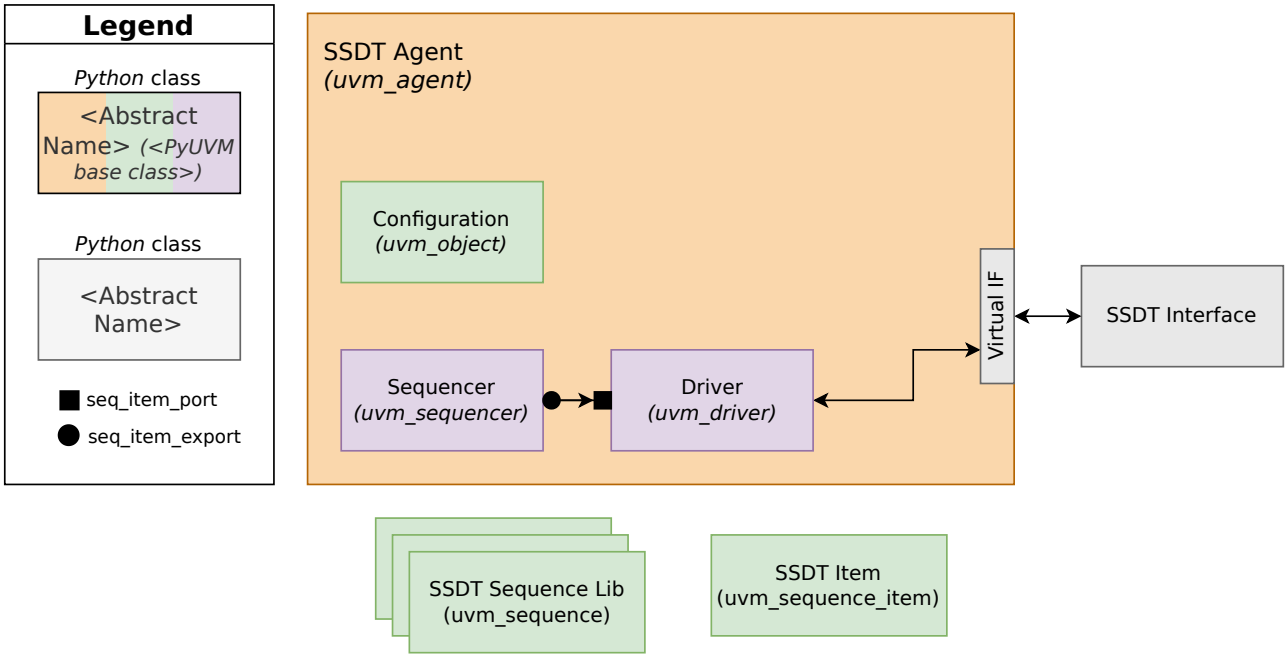


Fig. 4.1: *sSDT UVC* diagram.

The UVM components described in the Figure 4.1 can be found in the locations defined in the Table 4.2. The *UVC* is always available in the respective exercise directory under `<exercise>/tb/uvc/ssdt`. Consider for the `<ROOT_sSDT>` the following paths, `<exercise>/tb/uvc/ssdt`.

Component	Path
SSDT Agent	`<ROOT_sSDT>/uvc_ssdt_agent.py`
Configuration	`<ROOT_sSDT>/uvc_ssdt_config.py`
Driver	`<ROOT_sSDT>/uvc_ssdt_driver.py`
Sequencer	`uvm_sequencer` base class
SSDT Interface	`<ROOT_sSDT>/uvc_ssdt_interface.py`
SSDT Sequence Lib	`<ROOT_sSDT>/uvc_ssdt_sequence_lib.py`
SSDT Item	`<ROOT_sSDT>/uvc_ssdt_seq_item.py`

Table 4.2: *sSDT* UVC files location.

A Appendix

`Note` The commands listed below assume that the user is in the testbench directory (e.g. `<ROOT>/sat_filter/src/tb`).

- `make`
  Run the tests. All tests found will be run with default values for DUT parameters values.
- `make COCOTB_TEST_MODULES=<test-name>`
  Run a specific test. It will only simulate the test named `<test-name>` with the default values for the DUT parameters.
  e.g. `make COCOTB_TEST_MODULES=test_sat_filter_default_seq`.
- `make clean`
  Deleting all build and log files associated with the tests.
- `make coverage`
  Merge all tests coverage results and view the output in `PyUCIS-viewer` tool.
- `pyucis-viewer sim_build/<test-name>_cov.txt`
  Visualize the coverage report for a particular test case using the `PyUCIS-viewer` tool.
  e.g. `pyucis-viewer sim_build/test_sat_filter_default_seq.xml`
- `gtkwave sim_build/<waveform-name>.vcd`
  Visualize the waveform using the `GTKWave` tool.
  e.g. `gtkwave sim_build/sat_filter_waves.vcd`

.